

드림핵 patch문제 풀이

우선 문제를 풀기 위해서는 이 검은색으로 덧칠된 부분이 어디인지 부터 확인해야 했는데 그걸 확인하려면 우선 우선 window에 전달되는 정보를 담는 WINPROC부분을 확인하고 그걸 초기화하는 함수 내부에 들어가서 분석하기로 했다.

```
11 LoadStringW(hInstance, 0x6Du, &ClassName, 100);
12 v11.cbSize = 80;
13 v11.style = 3;
14 v11.lpfnWndProc = (WNDPROC)sub_7FF6D4A932F0;
15 *(_QWORD *)&v11.cbClsExtra = 0;
16 v11.hInstance = hInstance;
```

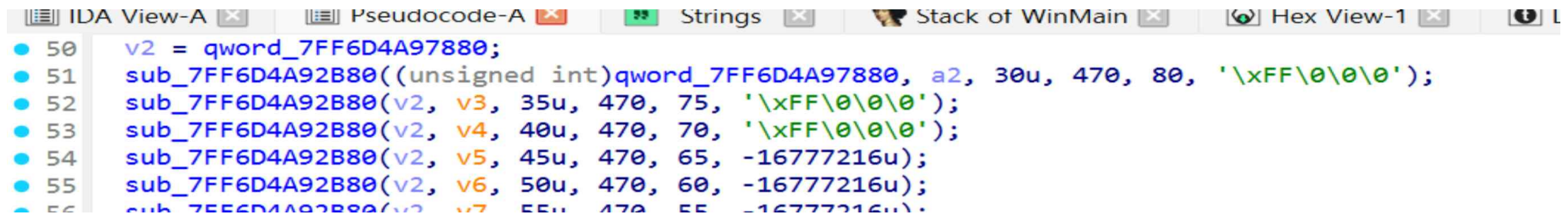
이 함수 내부에 들어와서 분석을 하니

```
11 return 0;
12 case 0xFu:
13     qword_7FF6D4A97910 = (__int64)BeginPaint(hWnd, &Paint);
14     v5 = ( QWORD *)GdiAlloc('\x10');
```

이렇게 BeginPaint함수가 있어서 우선 이 함수를 통해 전달되는 값을 확인하려고 x64dbg를 키고 분석했는데 흰색 배경을 그리는 데 사용하고 실제 글자 출력이나 가리기에는 사용이 되지 않는 듯 했다. 따라서 다른 부분에 중단점을 x64dbg를 통해서 걸어서 하나씩 확인해 보니 sub_7FF6D4A92C40함수 호출 후 화면에 DH{}와 같이 뜨는 걸

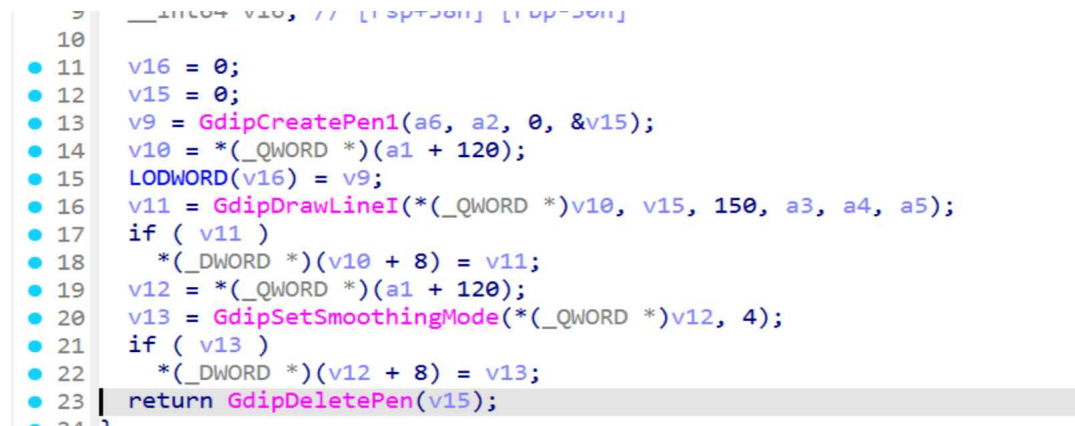
```
27 qword_7FF6D4A97918 = (__int64)v5;
28 ((void (*)(void))sub_7FF6D4A92C40)();
29 v6 = qword_7FF6D4A97918;
```

확인할 수 있었고 따라서 이 함수가 지금 화면에 무언갈 출력하는 함수라는 걸 알 수 있다. 그 정보를 토대로 이 함수 내부를 한 번 분석해 보면



```
IDA View-A | Pseudocode-A | Strings | Stack of WinMain | Hex View-1
50  v2 = qword_7FF6D4A97880;
51  sub_7FF6D4A92B80((unsigned int)qword_7FF6D4A97880, a2, 30u, 470, 80, '\xFF\0\0\0');
52  sub_7FF6D4A92B80(v2, v3, 35u, 470, 75, '\xFF\0\0\0');
53  sub_7FF6D4A92B80(v2, v4, 40u, 470, 70, '\xFF\0\0\0');
54  sub_7FF6D4A92B80(v2, v5, 45u, 470, 65, -16777216u);
55  sub_7FF6D4A92B80(v2, v6, 50u, 470, 60, -16777216u);
56  sub_7FF6D4A92B80(v2, v7, 55u, 470, 55, -16777216u);
```

이런 형식으로 무언갈 하고 있는데 여기에 나오는 sub_7FF6D4A92B80함수 안에 들어가서 무엇을 하는지를 자세히 파악해 보면 직선 형태의 판넬을 화면에 그린다는 것과 여기에 인자로 넘겨진 0xFF000000가 색깔 인자로 사용된



```
10  __int64 v10; // [rsp+50h] [rbp-50h]
11  v16 = 0;
12  v15 = 0;
13  v9 = GdipCreatePen1(a6, a2, 0, &v15);
14  v10 = *(_QWORD *)(a1 + 120);
15  LODWORD(v16) = v9;
16  v11 = GdipDrawLineI(*(_QWORD *)v10, v15, 150, a3, a4, a5);
17  if ( v11 )
18      *(_DWORD *)(v10 + 8) = v11;
19  v12 = *(_QWORD *)(a1 + 120);
20  v13 = GdipSetSmoothingMode(*(_QWORD *)v12, 4);
21  if ( v13 )
22      *(_DWORD *)(v12 + 8) = v13;
23  return GdipDeletePen(v15);
```

다는 사실을 알 수 있었다. 그 다음에 ai한테 0xFF000000가 무슨 색이니 물어보니 검정색이라고 했고 이를 통해서 드디어 내가 플래그를 가리는 직선들을 찾았다는 걸 확인했다. 하지만 이를 좀 더 명확하게 하기 위해서 x64dbg에서 중단점을 설정하고 실제로 어떤 작업을 하는지 보니 검정색 직선을 그리면서 플래그를 가리고 있었다. 따라서 이걸 전부다 x64dbg에서 nop으로 처리하고 실행하니

DH{UPATCHED}

와 같이 플래그가 정상적으로 출력되는 것을 확인할 수 있었다.

배운점

이 문제는 엄청 어렵지는 않지만 단순한 정적 분석이 아닌 정적 분석과 동적 분석 그리고 그에 따른 패치가 이루어져야 하기 때문에 난이도가 실버이고 WINDOW함수에 대해서도 공부할 수 있는 시간이 되서 좋았다.